Diseñar software con agentes de IA

Guía teórica del proyecto «app local de tareas»

Cómo usar este material

Este libro no enseña a programar desde cero: enseña a **diseñar software con agentes de IA**, usando como pretexto una aplicación chica y real (un tablero de tareas local en Node.js).

Cada capítulo cubre una etapa de la ruta y responde tres preguntas:

1. **Qué concepto hay que entender** (la teoría).
2. **Por qué importa** (qué se rompe si no se entiende).
3. **Cómo se conecta con el proyecto** (dónde se aplica).

La **práctica no está acá**: se lleva con el proyecto, en el repositorio, siguiendo RUTA-APRENDIZAJE.md. Este material es la teoría que hace falta para no copiar código sin entenderlo.

Una advertencia sobre el orden. El objetivo declarado no es tener un tablero de tareas: es **poder diseñar software con agentes**. El tablero es el pretexto, y tiene dos virtudes: es lo bastante chico como para verlo entero, y lo bastante real como para que las decisiones importen.

Capítulo 0 — El entorno

La idea central

Un programa no corre en el aire. Corre sobre un **intérprete**, dentro de un **sistema operativo**, con **permisos** y **variables** que no controlás del todo. La primera etapa no es escribir código: es **saber sobre qué terreno estás parado**.

Los conceptos

Intérprete. Node.js no compila a un ejecutable: lee el código y lo ejecuta línea por línea. El mismo archivo .js puede comportarse distinto según la versión del intérprete. Por eso la versión no es un detalle: es parte del entorno.

Versión. Un programa escrito para Node 22 puede fallar en Node 18 sin avisar. La versión no se supone: se consulta (node --version). Y una vez consultada, se anota, porque es un hecho del entorno que condiciona todo lo demás.

PATH. El PATH es la lista de carpetas donde el sistema busca los comandos. Cuando escribís node, el sistema recorre esa lista hasta encontrar un ejecutable con ese nombre. Si hay dos,

gana el primero. Esto explica por qué un comando que "funcionaba" deja de funcionar: cambió el orden de la lista, no el comando.

Política de ejecución (PowerShell). Windows no ejecuta cualquier script por defecto. En PowerShell, la política de ejecución puede bloquear un archivo .ps1 —el caso real de este proyecto: npm.ps1 está bloqueado, y por eso hay que usar npm.cmd. No es un error de instalación: es una decisión de seguridad del sistema, y se esquiva usando el archivo correcto.

Por qué importa

Este es el capítulo donde se aprende la lección que atraviesa todo el libro: verificar ejecutando es distinto de suponer.

Los tres datos de esta etapa se podían *suponer* y estar equivocado:

- Suponer que Node está instalado: puede no estarlo, o estar en otra versión.
- Suponer que node:sqlite requiere un flag experimental: en Node 22 **no lo requiere** — se supo al ejecutarlo.
- Suponer que npm funciona: falla por la política de ejecución, y el error **solo aparece al correrlo**.

Cómo se conecta con el proyecto

Acá no hay nada que construir: hay que **dejar registrado el entorno verificado** (versión de Node, cómo se invoca npm, qué módulos están disponibles) en DECISIONES.md. Ese registro es lo que permite que, dentro de tres meses, alguien (una persona o un agente) no tenga que adivinar por qué un comando falla.

Capítulo 1 — El servidor mínimo

La idea central

Una aplicación web son **dos programas hablando por un canal**. Entender el canal —y no solo la librería que lo envuelve— es lo que permite después elegir con criterio.

Los conceptos

Proceso. Un programa en ejecución. Tiene memoria propia, un identificador (PID) y un ciclo de vida: arranca, corre, termina. Cuando "levantás el servidor", estás creando un proceso que se queda esperando.

Puerto. Un número que identifica **a qué programa** dentro de la máquina llega una conexión. La dirección IP identifica la máquina; el puerto, al programa. Por eso dos servidores no pueden usar el mismo puerto a la vez: el sistema no sabría a cuál entregar la conexión.

localhost. La dirección que se refiere a la propia máquina. Cuando el servidor escucha en localhost, **no es accesible desde afuera**: solo desde el mismo equipo. Esto no es una limitación a superar, es la decisión de diseño de este proyecto (la app corre solo acá, sin nube).

Ciclo petición/respuesta. El modelo de HTTP: el cliente pide, el servidor responde, y la conexión se cierra. A diferencia de una conversación, cada petición es **independiente**: el servidor no recuerda la anterior. Esta característica —no tener estado— es lo que obliga a guardar todo en algún lado (el tema del Capítulo 2).

Método y ruta. Una petición HTTP tiene dos coordenadas: **qué** querés hacer (método: GET, POST…) y **sobre qué** (ruta: /api/tareas). Juntas identifican la operación.

Archivo estático vs. respuesta generada. Un servidor puede hacer dos cosas muy distintas: devolver un archivo tal cual está en disco (estático), o **construir** la respuesta en el momento (generada). La diferencia importa porque tienen riesgos distintos: el estático puede servirse mal (exponer un archivo que no correspondía), y el generado puede ser lento o inseguro según lo que haga.

Código de estado. Un número que acompaña la respuesta y dice **cómo salió**. 200 es "salió bien", 404 es "no existe", 500 es "se rompió del lado del servidor". No es decoración: es parte del contrato (Capítulo 3).

JSON como formato de intercambio. Un texto con estructura, legible por humanos y por programas. Se eligió sobre otros formatos porque es simple, estándar y no necesita herramientas para leerse.

Por qué importa

Sin express primero. La tentación es empezar con un framework. El problema es que un framework **esconde** el ciclo petición/respuesta detrás de una API cómoda, y entonces no se entiende qué pasa cuando algo falla. Escribir el servidor con el módulo node:http una vez —y ver el req y el res crudos— hace que después el framework sea una comodidad, no una caja negra.

Cómo se conecta con el proyecto

Esta etapa construye un servidor que sirve un index.html y responde en /api/salud con {"ok":true}.

La verificación es la parte importante: hay que probarlo con curl o Invoke-RestMethod, **no con el navegador**. El navegador "arregla" errores (muestra una página amable en vez del error crudo) y no muestra el código de estado con claridad. **Si no podés mostrar la respuesta cruda, no está verificado.**

Capítulo 2 — El modelo de datos

La idea central

Antes de guardar algo hay que decidir **qué es**. El modelo de datos es esa decisión, escrita de forma que la máquina pueda hacerla cumplir.

Los conceptos

Entidad. Algo sobre lo que guardás información, con identidad propia. En este proyecto: una **tarea**.

Atributo. Una propiedad de la entidad: título, estado, fecha de creación. Cada atributo tiene un **tipo** (texto, número, fecha), y el tipo es una promesa sobre qué operaciones tienen sentido.

Clave primaria. El atributo que identifica **inequívocamente** a cada fila. Dos tareas pueden llamarse igual; no pueden tener el mismo id.

Por qué esto es central en este proyecto: *en los archivos de texto del cuartel general, una tarea* ***no tiene identidad****. Es su línea de texto, y su posición en la lista. Si se reordena la lista, la tarea "cambia". La base de datos resuelve eso: le da un id.*

Clave foránea. Un atributo que apunta a la clave primaria de otra tabla. Es la forma de decir "esta tarea pertenece a este tablero" sin duplicar los datos del tablero.

Restricción. Una regla que la base hace cumplir: "el título no puede estar vacío", "el estado debe ser uno de estos cinco". Una restricción bien puesta es **una decisión escrita en un lugar que la máquina respeta**, en vez de una convención que cada programa debe recordar.

Invariante. Algo que debe ser cierto **siempre**, en todo momento válido. Ejemplo: "una tarea no puede estar en dos columnas a la vez".

Estado válido e inválido. Si el estado de una tarea es una lista cerrada, entonces "en progreso" es válido y "haciéndose" no. Definir la lista **antes** evita que cada parte del sistema invente su propio vocabulario.

Esquema. La descripción formal de las tablas y sus relaciones. Es el plano.

Migración. El cambio versionado del esquema. Un esquema **nunca** cambia "a mano": cambia con una migración, que es un paso escrito, reproducible y reversible. Esto importa cuando hay datos reales: si el esquema cambia sin migración, los datos viejos quedan huérfanos.

Por qué importa

El orden correcto es: modelo primero, código después. Escribir el INSERT antes de decidir qué campos tiene una tarea es la causa más común de tener que rehacer todo.

Y hay una razón práctica: en este proyecto la persistencia arranca en **JSON** y después pasa a **SQLite**, detrás de la misma interfaz. Eso solo funciona si el modelo está claro: el cambio de motor no debería tocar el resto del sistema.

Cómo se conecta con el proyecto

Antes de escribir una tabla hay **cinco preguntas** que responder, y sus respuestas son las decisiones del modelo:

4. ¿Qué es una tarea? ¿Qué campos tiene? ¿Cuáles son obligatorios?
5. ¿Qué estados puede tener? (la lista cerrada, no "lo que se te ocurra")
6. ¿Un tablero tiene columnas fijas o configurables?
7. ¿Se borra una tarea o se archiva?
8. ¿Guarda historial de movimientos? — **esta decide si un UPDATE alcanza o hace falta una tabla de eventos.**

La verificación de la etapa es tan importante como la construcción: **insertar a mano datos inválidos y comprobar que la base los rechaza**. Si la base acepta cualquier cosa, las restricciones no están puestas.

Capítulo 3 — El contrato

La idea central

Dos programas que se hablan necesitan acordar **cómo** se hablan. Ese acuerdo se llama contrato, y su valor está en que **permite que un lado cambie sin romper el otro**.

Los conceptos

Interfaz vs. implementación. La interfaz es **qué** promete hacer algo; la implementación es **cómo** lo hace. GET /api/tareas promete devolver las tareas; si las lee de un JSON o de SQLite es un detalle que el cliente no debería conocer.

Contrato estable. Un acuerdo que no cambia aunque cambie el interior. Es lo que permite que la base pase de JSON a SQLite sin tocar la interfaz.

Validación de entrada. Comprobar que lo que llega es lo que se esperaba. Un POST sin título no es un error del cliente que hay que ignorar: es una entrada inválida que hay que **rechazar con un código de estado claro**.

Idempotencia. Una operación es idempotente si **repetirla da el mismo resultado que hacerla una vez**. GET y DELETE lo son; POST no (dos veces crea dos tareas). Esto no es teoría: es lo que decide si un reintento automático es seguro o duplica datos.

Errores como parte de la interfaz. Un error no es una excepción al contrato: **es parte del contrato**. Si GET /api/tareas/999 devuelve 404 con un JSON explicando, el cliente sabe qué pasó. Si devuelve un HTML de error, rompe el flujo.

Por qué importa

La lección de fondo de esta etapa: el contrato es lo que permite que **la interfaz y los agentes cambien sin romperse**. Si el contrato no está escrito, cada consumidor adivina.

Y con agentes esto se multiplica: un agente que no tiene el contrato escrito **va a inventar** un formato. El contrato explícito es contexto reutilizable; la convención tácita es una alucinación esperando su turno.

Cómo se conecta con el proyecto

Esta etapa construye la API: GET /api/tareas, POST /api/tareas, PATCH /api/tareas/:id, DELETE /api/tareas/:id.

El detalle que importa: cada endpoint necesita tres cosas escritas — qué recibe, qué devuelve cuando sale bien, y qué devuelve cuando sale mal. Sin las tres, el contrato está a medias.

Capítulo 4 — La interfaz

La idea central

La interfaz es la parte que **tiene estado en dos lugares a la vez**: en el navegador, para que se vea, y en el servidor, para que persista. Sincronizar esos dos estados es el problema central de este capítulo.

Los conceptos

DOM. El árbol de objetos que representa la página. No es "el HTML": es la estructura viva que el navegador construyó a partir del HTML, y que el código puede modificar.

Eventos. Acciones que ocurren (un clic, una tecla, una respuesta que llega) y a las que el código se **suscribe**. La diferencia entre un evento y una función normal: el evento lo dispara algo externo, cuando quiere.

Estado en el cliente vs. en el servidor. El cliente tiene una copia para mostrar; el servidor tiene la verdad. **Cuando los dos cambian por separado, divergen.** El objetivo del diseño es que el cliente sea una **proyección** del estado del servidor, no una fuente paralela.

Render. Convertir el estado en lo que se ve. La regla útil: **el render es una función del estado**, no una serie de modificaciones sueltas del DOM. Si el estado es correcto, el render lo refleja.

Por qué el estado se duplica y cómo se sincroniza. Porque el navegador no puede consultar el servidor en cada microsegundo: tiene su copia. Se sincroniza con una regla clara: **una sola dirección**. El servidor manda; el cliente pide de nuevo después de cada cambio.

Drag & drop como problema de estado (no de mouse). La tentación es pensarlo como "seguir el mouse". El problema real es otro: al soltar, hay que decidir **qué cambió en el modelo** ("la tarea 7 pasó a la columna 3") y eso es lo que se manda al servidor. El mouse es la superficie; el estado es el fondo.

Por qué importa

Si el estado se guarda solo en el cliente, se pierde al recargar. Eso no es un bug: es la consecuencia de haber puesto la fuente de verdad en el lugar equivocado.

Cómo se conecta con el proyecto

La etapa construye el tablero con columnas y las cuatro operaciones (crear, mover, editar, borrar).

La verificación es directa y despiadada: recargar la página. Si el estado se pierde, la persistencia no estaba donde creías. Y el proyecto usa **HTML + JS vanilla, sin framework**: no porque sea mejor, sino porque así se ven el DOM, los eventos y fetch sin magia que los esconda.

Capítulo 5 — La verificación

La idea central

Una prueba es **código que ejecuta el sistema y afirma que el resultado es el esperado**. Su valor no es "demostrar que anda": es **detectar cuándo deja de andar**.

Los conceptos

Prueba que ejecuta. Una prueba que no ejecuta el código no prueba nada. Revisar el archivo con la vista no es probar.

La lección ya aprendida en este equipo: *node --check detecta* ***sintaxis****, no errores de referencia. Un módulo puede pasar el chequeo y fallar al ejecutarse. Está anotado como pendiente del sistema, y aplica acá desde el principio.*

Aserción. La afirmación concreta que se comprueba: "esto es igual a aquello". Si la aserción no se cumple, la prueba falla.

Prueba de integración. No comprueba una función aislada, sino **varias piezas juntas**: se levanta la base en memoria, se ejecuta la API, y se afirma el resultado.

Caso borde. La entrada rara: la lista vacía, el id que no existe, el texto larguísimo. Es donde suelen estar los errores, porque es la entrada que nadie probó a mano.

Regresión. Un error que **vuelve**: algo que funcionaba y dejó de funcionar por un cambio posterior. La prueba existe justamente para que eso se detecte en el momento, no en producción.

Por qué importa

Una prueba que nunca falla no prueba nada. Por eso la etapa exige el paso incómodo: **romper el código a propósito** y comprobar que la prueba lo detecta. Si al romperlo la prueba sigue en verde, la prueba es decorativa.

Cómo se conecta con el proyecto

Se construye un archivo de pruebas que levanta la base **en memoria**, ejecuta la API y afirma resultados. Node trae el runner incorporado (node:test, estable desde Node 20), así que **no hace falta instalar nada**.

Capítulo 6 — La integración con agentes

La idea central

Un agente de IA no es un usuario especial: es **otro programa**, que lee texto y produce texto. La diferencia es que **no completa lo que falta**: si el formato es ambiguo, inventa. Por eso esta etapa es, entera, sobre **hacer explícito el formato**.

Los conceptos

Interfaz de línea de comandos (CLI). Un programa con el que se habla escribiendo un comando. Es la forma más simple y estable de que un agente ejecute algo: no necesita una pantalla, solo una entrada y una salida.

Salida legible por máquina. Lo contrario de "una salida linda para humanos". Si el comando imprime una tabla con bordes decorativos, un agente tiene que adivinar cuáles caracteres son datos. Si imprime JSON, no adivina nada.

Formato estable. El formato no cambia sin aviso. Un agente que lee un formato que cambió no falla con elegancia: **lee mal y decide peor**.

Contrato entre un programa y un agente. El mismo concepto del Capítulo 3, pero con una diferencia de riesgo: un humano ve una salida rara y pregunta; un agente **sigue adelante con lo que entendió**.

Por qué el texto libre no es base de datos. Un archivo en prosa con listas, tablas y análisis entremezclados **se puede leer** pero no se puede automatizar sin riesgo. Separar lo estructurado de lo narrativo es una decisión de arquitectura, no una preferencia de estilo.

Por qué importa

Es el **objetivo real del proyecto**. Todo lo anterior —servidor, datos, contrato, interfaz, pruebas— existe para llegar acá con algo que un agente pueda usar sin adivinar.

Cómo se conecta con el proyecto

Se construye el comando que los presets gabinete y rendimiento van a poder correr, y el archivo de proyección que van a poder leer.

La verificación es el cierre del círculo: abrir una sesión con cada preset y comprobar que **leen y escriben sin ayuda**. Si hay que explicarle el formato al agente en la conversación, el formato no estaba explícito.

Cierre — Los principios que atraviesan todo

Estos once puntos no son de una etapa: son lo que se aprende **con** el proyecto. Cada uno tiene un caso real que lo originó.

9. **Leer antes de opinar.** El código que cambió hay que leerlo, no recordarlo. Un documento viejo y un archivo generado pueden contradecirse, y **solo se descubre leyendo los dos**.
10. **Verificar ejecutando.** "Debería funcionar" es una hipótesis. Un módulo puede no estar disponible, o requerir un flag: se sabe recién al correrlo.
11. **Una sola fuente de verdad.** Es el modo de falla más común y el más caro. Dos listas que se sincronizan a mano divergen en dos semanas, y desde ahí **no sabés cuál es la buena**.
12. **El estado estructurado y la narrativa no van en el mismo archivo.** Un archivo con listas *y* prosa *y* tablas se puede leer, pero no se puede automatizar sin riesgo. Separarlos es una decisión de arquitectura.
13. **Contratos explícitos.** Cuanto más explícito el formato, menos adivina el agente. Un esquema escrito es contexto reutilizable; una convención tácita es una alucinación esperando.
14. **Alcance acotado.** Un cambio chico produce un diff revisable. Un cambio grande produce un diff que **se aprueba sin leer**, y aprobar sin leer es no tener control.
15. **El contexto del agente es un recurso finito.** Por eso existen los archivos de instrucciones y las skills: son memoria externa que se carga solo cuando hace falta. **Escribir para un agente es escribir documentación operativa.**
16. **Puntos de control humanos.** Confirmar, publicar y manejar credenciales lo decide una persona. Un agente que lo hace solo te saca la última revisión que tenías.
17. **Verificación independiente.** El que revisa no debería ser el que escribió. Con agentes esto es literal: una instancia escribe, otra revisa, y la segunda no ve las intenciones de la primera —solo el resultado.
18. **Preguntar antes de destruir.** Reorganizar, mover o borrar pide confirmación. Vale para un agente y vale para un script.
19. **Lo que no se puede reproducir, no está terminado.** Si para saber si algo anda hay que "abrir la app y fijarse", no hay forma de saber si el cambio de ayer lo rompió.

Una última cosa: el método

El método de trabajo de este proyecto —**el agente propone, vos escribís, el agente revisa** — no es una preferencia pedagógica. Es una consecuencia de lo anterior.

Un agente que escribe todo el código **te deja sin entender el sistema que vas a mantener**, y sin la capacidad de revisar lo que el agente hace después. El objetivo no es la app: es la capacidad de diseñar la próxima.

Material generado para el proyecto «app local de tareas». La práctica se lleva en el repositorio, según RUTA-APRENDIZAJE.md. Las fuentes de consulta están en BIBLIOGRAFIA.md.